

Library Management System

A Relational Database Application — MySQL Implementation

Database Management Systems · Capstone Project

Project Report & Technical Documentation

Backend: MySQL 8.0 | Frontend: HTML / CSS / JavaScript + Node (Express)

Table of Contents

1. Project Overview & Objectives
2. System Architecture (3-tier)
3. Entity–Relationship Design
4. Relational Schema & Line-by-Line Explanation ([01_schema.sql](#))
5. Normalization — 1NF → 2NF → 3NF (with proof)
6. Data Dictionary
7. Sample Data ([02_data.sql](#))
8. Views ([03_views.sql](#)) — explained
9. Advanced SQL Queries ([04_queries.sql](#)) — explained
10. Triggers, Procedures & Transactions ([05_procedures.sql](#)) — explained
11. Frontend & Backend Integration
12. Testing
13. DBMS Concepts Explained (full reference)
14. What We Built & How to Run It (Docker)
15. Viva Questions & Answers

1. Project Overview & Objectives

This project is a complete **Library Management System** built on a normalized MySQL relational database. It models the real-world operations of a library: cataloguing books, tracking every physical copy, registering members and staff, issuing and returning books, charging fines for late returns, and handling reservations.

Objectives

- Design a relational schema in **Third Normal Form (3NF)** with full integrity constraints.
- Implement all relationship types — **1:M**, **M:N**, **recursive**, and **1:1**.
- Demonstrate advanced SQL: joins, subqueries, aggregates, **GROUP BY/HAVING**, views, indexes.
- Move integrity logic into the database via **triggers, stored procedures, and transactions**.
- Provide a working **frontend connected to the live database**.

Why a library system?

It naturally contains every relationship type the course requires: a book has many copies (1:M), a book has many authors and an author writes many books (M:N), a category can contain sub-categories (recursive), and each overdue loan produces at most one fine (1:1).

2. System Architecture (3-tier)

A web browser **cannot** connect to MySQL directly (no database driver, and the browser security sandbox forbids raw TCP to a DB). Every real database application therefore uses a **3-tier architecture**:

```
TIER 1 – Presentation      TIER 2 – Application      TIER 3 – Data
Browser (HTML/CSS/JS)  ---> Express API (Node.js)  ---> MySQL (library_db)
    fetch('/api/...')      calls views & stored      tables, views,
    renders tables          procedures, returns JSON    triggers, procedures
```

- **Presentation:** vanilla HTML/CSS/JS — renders the dashboard, tables, and forms.
- **Application:** Express server exposes REST endpoints; it never builds business rules itself — it calls the database's views and stored procedures.
- **Data:** MySQL holds the schema and *all* integrity logic (constraints + triggers), so the database stays correct even if accessed without the app.

3. Entity–Relationship Design

The system has **11 entities**. Two design decisions are central:

- **book vs. book_copy:** `book` is the *title* (one row per ISBN); `book_copy` is each *physical copy*. This lets one title have many copies, each with its own status (Available / Borrowed / Lost).
- **book_author junction:** resolves the many-to-many between books and authors.

Relationships & Cardinality

Relationship	Type	Implemented by	Participation
Publisher → Book	1 : M	<code>book.publisher_id</code>	Book total
Category → Book	1 : M	<code>book.category_id</code>	Book total
Category → Category	1 : M recursive	<code>category.parent_id</code> (self-FK)	both partial
Book ↔ Author	M : N	<code>book_author</code> junction	both partial
Book → BookCopy	1 : M	<code>book_copy.book_id</code>	Copy total
BookCopy → Borrowing	1 : M	<code>borrowing.copy_id</code>	Borrowing total
Member → Borrowing	1 : M	<code>borrowing.member_id</code>	Borrowing total
Staff → Borrowing	1 : M	<code>borrowing.issued_by</code>	Borrowing partial
Borrowing → Fine	1 : 1 (0..1)	<code>fine.borrow_id</code> UNIQUE	Fine total
Book / Member → Reservation	1 : M	<code>reservation</code> FKs	Reservation total

The crow's-foot ER diagram is in `docs/ER_DIAGRAM.md` (renders visually in VS Code / GitHub).
Paste that image here in the final report.

4. Relational Schema — Line by Line

Below, each table from `01_schema.sql` is shown with an explanation of every clause.

4.0 Database creation

```
DROP DATABASE IF EXISTS library_db;
CREATE DATABASE library_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE library_db;
```

- `DROP ... IF EXISTS` — removes an old copy so the script can be re-run cleanly.
- `utf8mb4` — full Unicode (supports any language / emoji in titles).
- `USE` — makes `library_db` the active database for all following statements.

4.1 publisher parent of book

```
CREATE TABLE publisher (
  publisher_id INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(120) NOT NULL UNIQUE,
  address     VARCHAR(200),
  city        VARCHAR(60),
  country     VARCHAR(60) DEFAULT 'India',
  contact_email VARCHAR(120),
  phone       VARCHAR(20)
) ENGINE=InnoDB;
```

- `publisher_id INT AUTO_INCREMENT PRIMARY KEY` — surrogate key; MySQL assigns 1,2,3... automatically. The PK guarantees each row is uniquely identifiable.
- `name ... NOT NULL UNIQUE` — a publisher must have a name, and no two publishers can share one.
- `country ... DEFAULT 'India'` — if no value is supplied, MySQL stores 'India'.
- `ENGINE=InnoDB` — the storage engine that supports **foreign keys** and **transactions** (required for this project).

4.2 category recursive

```
CREATE TABLE category (
  category_id INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(80) NOT NULL UNIQUE,
  parent_id   INT NULL,
  CONSTRAINT fk_category_parent
    FOREIGN KEY (parent_id) REFERENCES category(category_id)
    ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB;
```

- `parent_id` references **the same table** — this is the **recursive relationship**. "Physics" can have `parent_id` pointing to "Science".
- `NULL` parent = a top-level category.
- `ON DELETE SET NULL` — if a parent category is deleted, its children become top-level instead of being deleted.
- `ON UPDATE CASCADE` — if a parent's id changes, children update automatically.

4.3 author

```
CREATE TABLE author (
  author_id   INT AUTO_INCREMENT PRIMARY KEY,
  first_name  VARCHAR(60) NOT NULL,
  last_name   VARCHAR(60) NOT NULL,
  nationality  VARCHAR(60),
  birth_year  SMALLINT,
  CONSTRAINT chk_birth_year CHECK (birth_year IS NULL OR birth_year BETWEEN 1000 AND 2025)
) ENGINE=InnoDB;
```

- `CHECK (...)` — a **domain constraint**: a birth year, if given, must be a sane 4-digit year. The database rejects bad data even if the app forgets to validate.
- `SMALLINT` — a small integer type, enough for a year (saves space).

4.4 book M:1 publisher, category

```
CREATE TABLE book (
  book_id          INT AUTO_INCREMENT PRIMARY KEY,
  isbn             CHAR(13) NOT NULL UNIQUE,
  title            VARCHAR(200) NOT NULL,
  publisher_id     INT NOT NULL,
  category_id      INT NOT NULL,
  edition          VARCHAR(20),
  publication_year SMALLINT,
  language         VARCHAR(30) DEFAULT 'English',
  price            DECIMAL(8,2) CHECK (price >= 0),
  CONSTRAINT fk_book_publisher FOREIGN KEY (publisher_id)
    REFERENCES publisher(publisher_id) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_book_category FOREIGN KEY (category_id)
    REFERENCES category(category_id) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB;
```

- `isbn CHAR(13) ... UNIQUE` — fixed 13-char ISBN; UNIQUE prevents duplicates.
- `price DECIMAL(8,2)` — exact money type (8 digits, 2 after the point). Never use FLOAT for money — it rounds incorrectly.
- Two **foreign keys** link each book to exactly one publisher and one category (the 1:M side).
- `ON DELETE RESTRICT` — you cannot delete a publisher/category while books still reference it. This protects referential integrity.

4.5 book_author M:N junction

```
CREATE TABLE book_author (
  book_id          INT NOT NULL,
  author_id        INT NOT NULL,
  author_role      VARCHAR(30) DEFAULT 'Author',
  PRIMARY KEY (book_id, author_id),
  CONSTRAINT fk_ba_book FOREIGN KEY (book_id) REFERENCES book(book_id) ON DELETE CASCADE,
  CONSTRAINT fk_ba_author FOREIGN KEY (author_id) REFERENCES author(author_id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

- This is the **junction (bridge) table** that turns a many-to-many into two one-to-many links.
- `PRIMARY KEY (book_id, author_id)` — a **composite key**; the same book-author pair can appear only once.
- `ON DELETE CASCADE` — if a book or author is deleted, its link rows vanish too (a link to a non-existent book is meaningless).

4.6 book_copy physical inventory

```
CREATE TABLE book_copy (
  copy_id      INT AUTO_INCREMENT PRIMARY KEY,
  book_id      INT NOT NULL,
  barcode      VARCHAR(30) NOT NULL UNIQUE,
  shelf_location VARCHAR(30),
  status       ENUM('Available','Borrowed','Reserved','Lost','Damaged')
              NOT NULL DEFAULT 'Available',
  acquired_date DATE,
  CONSTRAINT fk_copy_book FOREIGN KEY (book_id) REFERENCES book(book_id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

- `ENUM(...)` — restricts `status` to a fixed set of values; anything else is rejected.
- `barcode ... UNIQUE` — each physical copy has its own scannable code.

4.7 member · 4.8 staff

```
CREATE TABLE member (
  member_id    INT AUTO_INCREMENT PRIMARY KEY,
  first_name   VARCHAR(60) NOT NULL,
  last_name    VARCHAR(60) NOT NULL,
  email        VARCHAR(120) NOT NULL UNIQUE,
  phone        VARCHAR(20), address VARCHAR(200),
  membership_type ENUM('Student','Faculty','Public') NOT NULL DEFAULT 'Student',
  join_date    DATE NOT NULL,
  is_active    BOOLEAN NOT NULL DEFAULT TRUE
) ENGINE=InnoDB;
```

- `email ... UNIQUE` — used as the natural identifier; no duplicate members.
- `is_active BOOLEAN` — supports a **soft delete** (deactivate rather than erase).
- `staff` is structured the same way and records who issued each loan.

4.9 borrowing

borrow / return transaction

```
CREATE TABLE borrowing (
  borrow_id INT AUTO_INCREMENT PRIMARY KEY,
  copy_id INT NOT NULL,
  member_id INT NOT NULL,
  issued_by INT, -- staff (nullable)
  borrow_date DATE NOT NULL,
  due_date DATE NOT NULL,
  return_date DATE NULL, -- NULL = still on loan
  CONSTRAINT fk_borrow_copy FOREIGN KEY(copy_id) REFERENCES book_copy(copy_id) ON DELETE RESTRICT,
  CONSTRAINT fk_borrow_member FOREIGN KEY(member_id) REFERENCES member(member_id) ON DELETE RESTRICT,
  CONSTRAINT fk_borrow_staff FOREIGN KEY(issued_by) REFERENCES staff(staff_id) ON DELETE SET NULL,
  CONSTRAINT chk_due_after_borrow CHECK (due_date >= borrow_date),
  CONSTRAINT chk_return_after_borrow CHECK (return_date IS NULL OR return_date >= borrow_date)
) ENGINE=InnoDB;
```

- The central transaction table — it links a **copy**, a **member**, and the issuing **staff**.
- `return_date NULL` cleverly means "not yet returned" — every active-loan query filters on this.
- Two `CHECK` constraints stop illogical dates (due before borrow, returned before borrowed).
- `ON DELETE RESTRICT` on copy/member keeps loan history intact; staff uses SET NULL so deleting a staff member doesn't erase the loan.

4.10 fine 1:1 · 4.11 reservation

```
CREATE TABLE fine (
  fine_id INT AUTO_INCREMENT PRIMARY KEY,
  borrow_id INT NOT NULL UNIQUE, -- UNIQUE makes it 1:1 with borrowing
  amount DECIMAL(8,2) NOT NULL CHECK (amount >= 0),
  reason VARCHAR(100),
  paid BOOLEAN NOT NULL DEFAULT FALSE,
  paid_date DATE NULL,
  CONSTRAINT fk_fine_borrow FOREIGN KEY(borrow_id) REFERENCES borrowing(borrow_id) ON DELETE CASCADE
) ENGINE=InnoDB;
```

- `borrow_id ... UNIQUE` — a FK that is also UNIQUE creates a **one-to-one** relationship: one loan can have at most one fine.
- `reservation` records a member reserving a title, with an ENUM status (Pending / Fulfilled / Cancelled).

4.12 Indexes & ALTER

```
CREATE INDEX idx_book_title    ON book(title);
CREATE INDEX idx_copy_status  ON book_copy(status);
CREATE INDEX idx_borrow_due   ON borrowing(due_date);
-- ... plus indexes on category, member name, borrow member

ALTER TABLE member ADD COLUMN last_updated TIMESTAMP
    DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP;
```

- **Indexes** are like a book's index — they let MySQL find rows without scanning the whole table. We index columns used in searches (`title`) and filters (`status` , `due_date`). PK and UNIQUE columns are indexed automatically.
- The `ALTER TABLE` demonstrates schema modification — it adds an audit timestamp that updates itself whenever a member row changes.

5. Normalization — 1NF → 2NF → 3NF

Normalization removes redundancy and update/insert/delete anomalies. We start from a single "flat" table and decompose it step by step.

Unnormalized (UNF) — the bad design

```
LOAN(member_name, member_email, book_title, authors, publisher_name,  
      publisher_city, borrow_date, due_date, fine_amount)
```

Problems: authors is multi-valued; publisher city repeats for every book; if a book has no loan yet we can't store it; updating a publisher's city means editing many rows.

First Normal Form (1NF) — atomic values, no repeating groups

Rule: every cell holds one value. `authors` ("Cormen, Leiserson") is multi-valued, which breaks 1NF.

Fix: pull authors into their own `author` table and link via the `book_author` junction. Likewise, multiple physical copies become rows in `book_copy`.

Second Normal Form (2NF) — no partial dependency

Rule: (applies to tables with a composite key) every non-key column must depend on the *whole* key, not part of it. In `book_author` the key is `(book_id, author_id)`. If we had stored `book_title` here, it would depend only on `book_id` — a **partial dependency**. **Fix:** `book_title` lives in `book`; `book_author` keeps only `author_role`, which truly depends on both book and author.

Third Normal Form (3NF) — no transitive dependency

Rule: no non-key column may depend on another non-key column. In a flat `book` table holding `publisher_name, publisher_city`, the city depends on the publisher, not on the book — a **transitive dependency** (`book_id → publisher → city`). **Fix:** move publisher attributes into `publisher` and reference it with `publisher_id`. The same reasoning moves category data to `category` and fine data to `fine`.

Functional dependency	Lives in table
$\text{publisher_id} \rightarrow \text{name, city, country}$	publisher
$\text{category_id} \rightarrow \text{name, parent_id}$	category
$\text{book_id} \rightarrow \text{isbn, title, publisher_id, category_id, price}$	book
$\text{author_id} \rightarrow \text{first_name, last_name, nationality}$	author
$(\text{book_id}, \text{author_id}) \rightarrow \text{author_role}$	book_author
$\text{copy_id} \rightarrow \text{book_id, barcode, status}$	book_copy
$\text{borrow_id} \rightarrow \text{copy_id, member_id, dates}$	borrowing
$\text{borrow_id} \rightarrow \text{amount, paid (fine)}$	fine

Result: every non-key attribute depends on "the key, the whole key, and nothing but the key." No redundancy, no anomalies — the schema is in **3NF**.

6. Data Dictionary (key tables)

Full version in [docs/DATA_DICTIONARY.md](#) . Key = PK primary, FK foreign, UQ unique, NN not null.

book

Column	Type	Constraints	Description
book_id	INT	PK, AUTO_INCREMENT	Unique title id
isbn	CHAR(13)	NN, UQ	ISBN-13
title	VARCHAR(200)	NN	Title
publisher_id	INT	NN, FK→publisher	Publisher
category_id	INT	NN, FK→category	Category
price	DECIMAL(8,2)	CHECK ≥ 0	List price

borrowing

Column	Type	Constraints	Description
borrow_id	INT	PK	Loan id
copy_id	INT	NN, FK→book_copy	Copy loaned
member_id	INT	NN, FK→member	Borrower
due_date	DATE	NN, CHECK ≥ borrow_date	Due date
return_date	DATE	NULL = still out	Return date

7. Sample Data ([02_data.sql](#))

The database is populated with **~200 meaningful records** (well above the 100 minimum):

Table	Rows	Table	Rows
publisher	10	book_copy	40
category	12	member	20
author	20	staff	5
book	25	borrowing	30
book_author	~24	fine	7
reservation	8	Total	~200

Loans deliberately include **returned**, **active**, and **overdue** cases so that the overdue/fine queries and views return real results during the demo.

8. Views (`03_views.sql`)

A **view** is a saved query that behaves like a virtual table — it simplifies complex joins and is reused by both the queries and the frontend.

`vw_book_catalogue` — books with authors joined into one row

```
CREATE OR REPLACE VIEW vw_book_catalogue AS
SELECT b.book_id, b.isbn, b.title, c.name AS category, p.name AS publisher,
       b.price,
       GROUP_CONCAT(CONCAT(a.first_name, ' ', a.last_name)
                    ORDER BY a.last_name SEPARATOR ', ') AS authors
FROM book b
JOIN category c ON b.category_id = c.category_id
JOIN publisher p ON b.publisher_id = p.publisher_id
LEFT JOIN book_author ba ON b.book_id = ba.book_id
LEFT JOIN author a ON ba.author_id = a.author_id
GROUP BY b.book_id, b.isbn, b.title, c.name, p.name, b.price;
```

- `GROUP_CONCAT` merges a book's many authors into one comma-separated string.
- `LEFT JOIN` to authors keeps books that have no author recorded.

The other three views: **`vw_book_availability`** (copies total vs available, using `SUM(status='Available')`), **`vw_active_loans`** (un-returned loans with `DATEDIFF(CURDATE(), due_date)` as days overdue), and **`vw_outstanding_fines`** (unpaid fines summed per member).

9. Advanced SQL Queries (04_queries.sql)

9.1 Joins

```
-- Every borrowing with member, book and issuing staff (multi-table join)
SELECT br.borrow_id, CONCAT(m.first_name, ' ', m.last_name) AS member, b.title
FROM borrowing br
JOIN book_copy bc ON br.copy_id = bc.copy_id
JOIN book b      ON bc.book_id = b.book_id
JOIN member m    ON br.member_id = m.member_id;
```

We must hop `borrowing → book_copy → book` because loans reference a *copy*, and the title lives on `book`. A **self join** on `category` lists each sub-category beside its parent; a **LEFT JOIN** finds members who never borrowed.

9.2 Aggregates + GROUP BY / HAVING

```
-- Most-borrowed books; HAVING filters the grouped result
SELECT b.title, COUNT(br.borrow_id) AS times_borrowed
FROM book b
JOIN book_copy bc ON b.book_id = bc.book_id
JOIN borrowing br ON bc.copy_id = br.copy_id
GROUP BY b.book_id, b.title
HAVING COUNT(br.borrow_id) >= 2
ORDER BY times_borrowed DESC;
```

WHERE vs HAVING: `WHERE` filters individual rows *before* grouping; `HAVING` filters the *groups* after aggregation. You cannot use `COUNT()` in a `WHERE` clause — that's exactly why `HAVING` exists.

9.3 Nested / correlated subqueries

```
-- Books priced above the overall average (scalar subquery)
SELECT title, price FROM book
WHERE price > (SELECT AVG(price) FROM book);

-- Members who currently have an OVERDUE book (correlated subquery + EXISTS)
SELECT m.member_id, m.first_name FROM member m
WHERE EXISTS (SELECT 1 FROM borrowing br
              WHERE br.member_id = m.member_id
                 AND br.return_date IS NULL AND br.due_date < CURDATE());
```

- **Scalar subquery** — the inner query returns one value (the average) used in a comparison.

- **Correlated subquery** — the inner query references the outer row (`m.member_id`), so it re-evaluates per member. `EXISTS` is true if it finds any overdue loan.
- A **NOT IN** subquery finds books that were never borrowed.

10. Triggers, Procedures & Transactions (05_procedures.sql)

This moves business rules *into the database* so data stays correct no matter how it is accessed.

Trigger 1 — block issuing an unavailable copy

```
CREATE TRIGGER trg_borrow_before_insert
BEFORE INSERT ON borrowing FOR EACH ROW
BEGIN
    DECLARE v_status VARCHAR(20);
    SELECT status INTO v_status FROM book_copy WHERE copy_id = NEW.copy_id;
    IF v_status <> 'Available' THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Copy is not available';
    END IF;
END;
```

- **BEFORE INSERT ... FOR EACH ROW** — runs automatically just before each new loan row.
- **NEW.copy_id** — the value being inserted.
- **SIGNAL SQLSTATE '45000'** — raises a custom error that **aborts the insert**; the error message bubbles all the way up to the web UI.

Trigger 2 & 3 — auto status + auto fine

```
-- After a loan is created, mark the copy Borrowed
CREATE TRIGGER trg_borrow_after_insert AFTER INSERT ON borrowing FOR EACH ROW
    UPDATE book_copy SET status='Borrowed' WHERE copy_id = NEW.copy_id;

-- On return, free the copy and auto-create a fine if late
CREATE TRIGGER trg_borrow_after_update AFTER UPDATE ON borrowing FOR EACH ROW
BEGIN
    IF NEW.return_date IS NOT NULL AND OLD.return_date IS NULL THEN
        UPDATE book_copy SET status='Available' WHERE copy_id = NEW.copy_id;
        IF DATEDIFF(NEW.return_date, NEW.due_date) > 0 THEN
            INSERT INTO fine(borrow_id, amount, reason)
            VALUES(NEW.borrow_id, DATEDIFF(NEW.return_date, NEW.due_date)*10, 'Late return');
        END IF;
    END IF;
END;
```

OLD vs **NEW** : in an UPDATE trigger, **OLD** is the row before and **NEW** after. The condition "OLD return was NULL, NEW is set" detects the exact moment of return, so the fine is created only once, at ₹10 per day late.

Stored procedure + transaction

```

CREATE PROCEDURE sp_issue_book(IN p_copy_id INT, IN p_member_id INT,
                               IN p_staff_id INT, IN p_days INT, OUT p_borrow_id INT)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION BEGIN ROLLBACK; RESIGNAL; END;
  START TRANSACTION;
  INSERT INTO borrowing(copy_id,member_id,issued_by,borrow_date,due_date)
  VALUES(p_copy_id,p_member_id,p_staff_id,CURDATE(),
          DATE_ADD(CURDATE(),INTERVAL p_days DAY));
  SET p_borrow_id = LAST_INSERT_ID();
  COMMIT;
END;

```

- **Transaction** = all-or-nothing. `START TRANSACTION ... COMMIT` makes the changes permanent only if everything succeeds.
- The **EXIT HANDLER** catches any error, runs `ROLLBACK` (undo), and re-raises it — so a failed issue leaves the database untouched (ACID consistency).
- `IN` / `OUT` parameters pass data in and return the new `borrow_id`.
- A **stored function** `fn_member_balance(member_id)` returns a member's unpaid total.

11. Frontend & Backend Integration

The UI is plain HTML/CSS/JavaScript with six tabs (Dashboard, Books, Members, Active Loans, Issue Book, Fines). JavaScript calls the API with `fetch()` and renders the JSON as tables.

```
// frontend: ask the API for books, draw a table
const rows = await fetch('/api/books').then(r => r.json());

// backend (Express): read from the views and return JSON
app.get('/api/books', (req,res) =>
  pool.query(`SELECT * FROM vw_book_catalogue ...`)
    .then([rows] => res.json(rows)));

// issuing a book calls the stored procedure (integrity stays in the DB)
app.post('/api/loans', ... pool.query('CALL sp_issue_book(?,?,?,?,@bid)', [...]));
```

Because issue/return go through stored procedures, the triggers fire automatically — issuing an unavailable copy is rejected by the database and the error text is shown to the user. This is the **frontend–backend integration** the rubric asks for.

12. Testing

#	Test case	Expected result
1	Issue an Available copy	Loan created; copy → Borrowed (trigger)
2	Issue a Borrowed copy	Rejected by <code>trg_borrow_before_insert</code>
3	Return a copy after due date	Copy → Available; fine auto-created
4	Insert book with duplicate ISBN	Rejected by UNIQUE constraint
5	Insert price = -50	Rejected by CHECK (price ≥ 0)
6	Delete a publisher that has books	Rejected by ON DELETE RESTRICT
7	Search "Harry" in Books tab	Returns Harry Potter titles
8	Pay a member's fines	Fines marked paid; removed from list

13. DBMS Concepts Explained (full reference)

Every core concept used in this project, defined simply and tied to where it appears here.

13.1 DBMS, RDBMS and the relational model

- **DBMS** — software to store, manage and query data (e.g. MySQL).
- **RDBMS** — a DBMS where data is stored in **relations (tables)** of rows and columns, with relationships expressed through keys. MySQL is an RDBMS.
- **Relation / Tuple / Attribute** — a table / a row / a column.
- **Degree** = number of columns; **Cardinality** = number of rows.
- **Schema** — the structure (tables + columns + constraints); **Instance** — the actual data at a point in time.

13.2 Keys

Key	Meaning	Example in this project
Super key	Any set of columns that uniquely identifies a row	{book_id}, {book_id, title}
Candidate key	A minimal super key	book_id and isbn (both unique)
Primary key	The chosen candidate key; unique + not null	book_id
Alternate key	A candidate key not chosen as primary	isbn
Composite key	A key made of 2+ columns	book_author(book_id, author_id)
Foreign key	References a primary key in another table	book.publisher_id
Surrogate key	An artificial id (AUTO_INCREMENT) with no business meaning	all *_id columns

13.3 Integrity constraints

- **Entity integrity** — primary key cannot be NULL (every row is identifiable).
- **Referential integrity** — a foreign key must match an existing primary key (or be NULL). Enforced by all our FK constraints.
- **Domain integrity** — values fall in a valid range/type — our CHECK and ENUM constraints.
- **Key/Unique integrity** — UNIQUE on isbn, barcode, email.

13.4 SQL sub-languages

Category	Stands for	Commands	Used in
DDL	Data Definition	CREATE, ALTER, DROP, TRUNCATE	01_schema.sql
DML	Data Manipulation	INSERT, UPDATE, DELETE, SELECT	02_data, 04_queries
DCL	Data Control	GRANT, REVOKE	(privileges)
TCL	Transaction Control	COMMIT, ROLLBACK, SAVEPOINT	05_procedures.sql

13.5 Types of JOIN

- **INNER JOIN** — only rows matching in both tables (most queries here).
- **LEFT (OUTER) JOIN** — all left rows + matches; NULL where none — e.g. members who never borrowed.
- **RIGHT JOIN** — mirror of LEFT.
- **FULL OUTER JOIN** — all rows from both (MySQL emulates with UNION).
- **SELF JOIN** — a table joined to itself — category to its parent.
- **CROSS JOIN** — every combination (Cartesian product).

13.6 Subqueries

- **Scalar** — returns one value (price > AVG(price)).
- **Row / table** — returns rows used with IN / NOT IN.
- **Correlated** — references the outer query and re-runs per row (EXISTS overdue check).
- **Derived table** — a subquery in the FROM clause treated as a temporary table.

13.7 Aggregate functions & grouping

COUNT, SUM, AVG, MIN, MAX collapse many rows into one value. GROUP BY forms groups; HAVING filters those groups (unlike WHERE which filters rows before grouping).

13.8 Normalization — all forms

Form	Removes	Rule
1NF	Repeating groups / multi-valued cells	Atomic values only
2NF	Partial dependency	Non-key depends on the whole composite key
3NF	Transitive dependency	Non-key depends only on the key
BCNF	Remaining anomalies	Every determinant is a candidate key

Our schema satisfies 3NF (and effectively BCNF, since every determinant is a key). A **functional dependency** $X \rightarrow Y$ means X determines Y. **Anomalies** are problems in badly designed tables: *insert* (can't add a publisher with no book), *update* (change a city in many rows), *delete* (deleting the last loan loses the member).

13.9 ER model concepts

- **Entity** — a real-world object (Book, Member). **Weak entity** — depends on another for identity (a copy is meaningless without its book).
- **Attribute types** — simple, composite (name $\rightarrow$ first/last), derived (days_overdue, computed), multi-valued (authors $\rightarrow$ moved to its own table for 1NF).
- **Cardinality** — 1:1, 1:M, M:N (how many of each side relate).
- **Participation** — total (mandatory, e.g. every book needs a publisher) vs partial (optional).

13.10 Transactions & ACID

- **Atomicity** — all steps happen or none (our ROLLBACK handler).
- **Consistency** — constraints hold before and after.
- **Isolation** — concurrent transactions don't corrupt each other.
- **Durability** — once committed, data survives a crash (InnoDB + COMMIT).

13.11 Indexes, Views, Triggers, Procedures vs Functions

- **Index** — speeds up search at the cost of space + slower writes.
- **View** — a stored SELECT used like a virtual table; simplifies and reuses logic.
- **Trigger** — code that runs automatically on INSERT/UPDATE/DELETE; enforces rules in the DB.
- **Stored procedure** — a named block of SQL you CALL; may change data, can have IN/OUT params.
- **Stored function** — returns a single value and is used inside queries (`fn_member_balance`).

14. What We Built & How to Run It

14.1 Deliverables in this project

File / folder	What it contains
<code>01_schema.sql</code>	11 tables, all constraints, indexes, ALTER
<code>02_data.sql</code>	~200 sample records
<code>03_views.sql</code>	4 views
<code>04_queries.sql</code>	Advanced query demonstrations
<code>05_procedures.sql</code>	3 triggers, 3 procedures, 1 function, transactions
<code>frontend/</code>	Express API (server.js, db.js) + vanilla HTML/CSS/JS UI in <code>public/</code>
<code>docs/</code>	ER diagram, data dictionary, report outline, this PDF
<code>docker-compose.yml</code>	One-command MySQL setup that auto-loads the SQL

14.2 Architecture recap

```
Browser (HTML/CSS/JS) --> Express API (Node) --> MySQL (library_db)
  fetch('/api/...')          CALL procedures / views    tables + triggers
```

14.3 How to run (Docker)

```
# 1. start MySQL; it auto-loads schema, data, views, procedures
docker compose up -d

# 2. wait ~30s, then verify (should print 25)
docker exec -it library_mysql mysql -uroot -proot \
  -e "SELECT COUNT(*) FROM library_db.book;"

# 3. start the web app
cd frontend
npm start

# 4. open the browser
http://localhost:3000
```

Common mistake: opening the page via Live Server (`127.0.0.1:5500/...`) shows no data, because that address has no backend. The data only loads from `http://localhost:3000` , which is

served by the Node server connected to MySQL.

14.4 Features demonstrated in the UI

- **Dashboard** — live counts of books, members, active loans, outstanding fines.
- **Books** — searchable catalogue with availability (reads `vw_book_catalogue`).
- **Members** — list with active/inactive status.
- **Active Loans** — overdue highlighting + one-click Return (fires the auto-fine trigger).
- **Issue Book** — calls `sp_issue_book`; an unavailable copy is rejected by a trigger.
- **Fines** — outstanding totals per member with a Pay action.

15. Viva Questions & Answers

Q. What is normalization and why 3NF?

Organizing tables to remove redundancy and update/insert/delete anomalies. 3NF ensures every non-key attribute depends only on the primary key, so each fact is stored once.

Q. Where is the M:N relationship?

Between `book` and `author`, resolved by the junction table `book_author` with composite primary key `(book_id, author_id)`.

Q. Show a recursive relationship.

`category.parent_id` is a foreign key to `category.category_id`, so a category can be the child of another (Physics → Science).

Q. How is a 1:1 relationship created?

`fine.borrow_id` is a foreign key that is also `UNIQUE` — each loan can have at most one fine.

Q. Why separate `book` and `book_copy` ?

`book` stores title-level facts once (ISBN, author, price); `book_copy` stores each physical item with its own status. This avoids repeating title data per copy.

Q. Difference between `WHERE` and `HAVING`?

`WHERE` filters rows before grouping; `HAVING` filters groups after aggregation. Aggregate functions like `COUNT()` can only be used in `HAVING`.

Q. What is the difference between `DELETE`, `TRUNCATE` and `DROP`?

`DELETE` removes selected rows (can be rolled back); `TRUNCATE` removes all rows quickly and resets auto-increment; `DROP` removes the whole table/structure.

Q. What are ACID properties? Where do you use them?

Atomicity, Consistency, Isolation, Durability. Our `sp_issue_book` and `sp_return_book` wrap changes in a transaction with rollback, guaranteeing all-or-nothing.

Q. How do you enforce integrity without the application?

Constraints (PK, FK, UNIQUE, CHECK, ENUM, NOT NULL) plus triggers — e.g.

`trg_borrow_before_insert` blocks issuing an unavailable copy, and `trg_borrow_after_update` auto-creates a fine on late return.

Q. What is an index and what are its trade-offs?

A sorted structure that speeds up reads/searches on a column. Trade-off: it uses extra space and slightly slows inserts/updates because the index must be maintained.

Q. Why can't the browser talk to MySQL directly?

Browsers have no DB driver and the security sandbox forbids raw database connections, so a backend (here Express/Node) sits in between — the standard 3-tier architecture.

Q. What is a primary key vs a foreign key?

A primary key uniquely identifies a row within its table; a foreign key is a column that references a primary key in another table, enforcing referential integrity.

Q. What does ON DELETE CASCADE vs RESTRICT vs SET NULL do?

CASCADE deletes child rows with the parent; RESTRICT blocks deleting a parent that still has children; SET NULL clears the child's foreign key. We use CASCADE for junctions, RESTRICT to protect loan history, SET NULL for the optional issuing staff.

— End of Report —